Implementation of the Experimental Chatbot System

Joël Sieber
University of St.Gallen (HSG)

Supplementary Materials

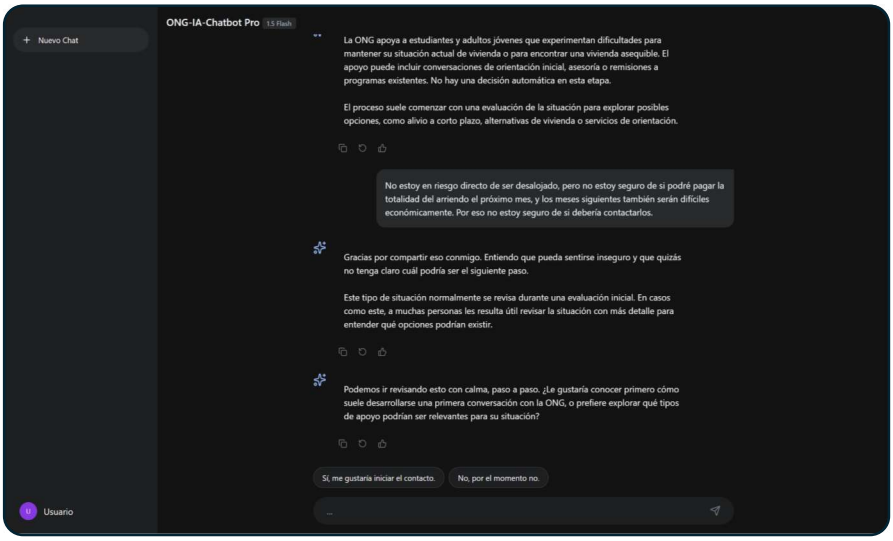
1. Overview

This document provides a detailed description of the technical implementation of the experimental chatbot used in this study. The primary objective was to balance strict internal validity with a realistic, ecologically valid interaction experience. To ensure full control over all verbal stimuli and to eliminate variance inherent in open-ended generative systems, the chatbot was implemented as a fully scripted, deterministic dialogue system. Within this framework, all participants assigned to a given experimental condition received identical chatbot responses. This approach was particularly important given the NGO context of the study, as it minimizes the risk of generating inappropriate or sensitive content. While the backend logic remained strictly deterministic, the frontend interface was designed to resemble contemporary conversational AI systems (e.g., ChatGPT, Gemini) in order to maintain a realistic user experience (see Figure 1).

The full source code for the experimental chatbot is hosted on the author's GitHub Repository, while the live experimental environment is accessible via Vercel.

Figure 1

User Interface of the Experimental Chatbot (Desktop Version)



2. Technical Architecture

The experimental environment was implemented as a *single-page application (SPA)* to ensure smooth transitions and low latency. The specific versions and libraries used are detailed in Table 1.

Table 1

System Specifications and Frameworks used

Component	*Implementation*	*Version / Detail*
Core Framework	React	v19.2.0
Build Tooling	Vite	v7.2.4
Styling Engine	Tailwind CSS	v3.4.17
Iconography	Lucide-React	v0.561.0
Repository	GitHub	Source code hosting
Deployment	Vercel Platform	Cloud-based hosting
State Persistence	Browser sessionStorage	chat_exp_v9_final_osf

To account for the high likelihood of mobile participation, the system implemented session-based state persistence. The application continuously stored the interaction state, including:

- message history
- current dialogue step
- experimental condition
- participant identifier (pid)
- interaction metrics
- session start timestamp

In the event of an unintended page reload, the system restored the interaction state from *sessionStorage*. This reduces the risk of data loss and allows participants to continue the interaction without restarting the experiment.

3. Experimental Flow and Condition Assignment

The experiment was integrated with *Qualtrics*, which handled participant recruitment and condition randomization. Participants were randomly assigned to one of four experimental conditions within Qualtrics. The assigned condition (cond) and participant ID (pid) were passed to the chatbot application via URL parameters. After completion of the chatbot interaction, participants were redirected back to Qualtrics. During this redirect, the application appended interaction data as URL parameters. Therefore, the structure displayed in Table 2 allows seamless integration of behavioral interaction data into the survey dataset.

Table 2

Data Transfer Variables

Direction	*Variable*	*Description*
Inbound (Qualtrics to App)	pid	Participant identifier
	cond	Assigned experimental condition (1–4)
Outbound (App to Qualtrics)	chat_duration	Total interaction time (seconds)
	rt_x	Reaction time per decision point (ms)
	choice_x	Selected option per step
	uptake	Final decision (binary)

4. Temporal Dynamics and Interaction Simulation

To avoid the perception of static or instantaneous system responses, the chatbot interaction was temporally structured to resemble contemporary conversational AI systems. Each chatbot response followed a staged process:

1. **Typing Simulation:** A short typing phase to simulate processing was displayed before each message using a CSS-animated three-dot indicator *(@keyframes bounce)*.
2. **Streaming Mechanism:** The messages were rendered at a fixed speed of 15 ms per character. A blinking terminal-style cursor *(.cursor-blink)* was appended to the text during this phase to enhance visual realism.
3. **Dynamic Reading Delay:** Response options remained hidden until the message was fully rendered and were dynamically calculated based on message length, as seen in Table 3.

Table 3

Interaction Timing Parameters

Parameter	*Value*	*Logic*
Streaming Speed	~15 ms / char.	Fixed rendering speed for chatbot messages
Minimum Delay	800 ms	Lower bound for reading time
Maximum Delay	4000 ms	Upper bound for reading time
Step 1 Delay	1200 ms	Fixed delay for initial interaction step
Delay Proportionality	15 ms / char.	Proportional scaling based on message length

The delay before response options became available was calculated as:

$$Delay = \max(800, \min(4000, length\ x\ 15))$$

This timing structure creates a more natural interaction rhythm and reduces the likelihood of rapid, non-deliberate response behavior.

5. Interface Design and Responsive Behavior

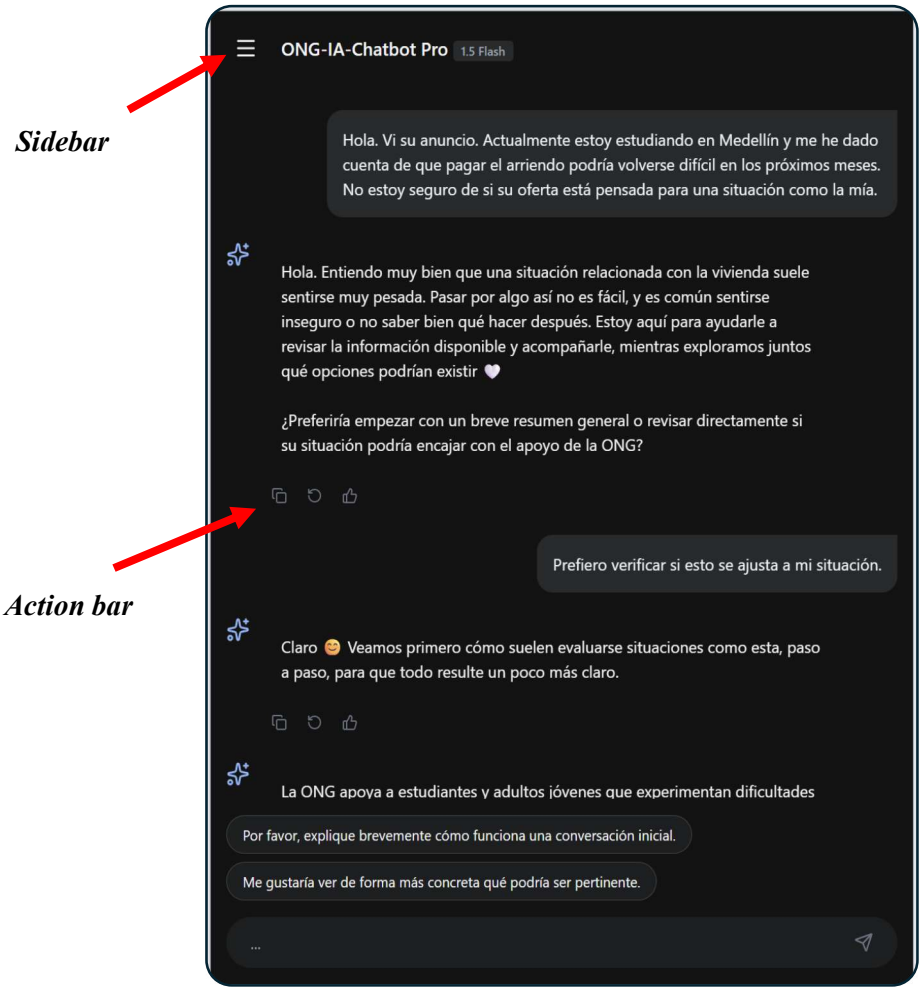
The interface adhered to modern "Dark Mode" design principles to enhance immersion, using: background (#131314), message containers (#1E1F20), and accent color (#8AB4F8).

The layout was fully responsive. On desktop devices, a sidebar navigation panel was displayed permanently, while on mobile devices it was hidden and accessible via a toggle button. To ensure visual consistency across various mobile browsers and to prevent layout shifts caused by browser UI elements (e.g., address bars), the layout was anchored using *Dynamic Viewport Height (100dvh)*.

Each chatbot message included a minimal, non-functional action bar (e.g., copy, regenerate, feedback icons) as shown in Figure 2. These elements were included purely for interface realism and had no effect on the experimental manipulation.

Figure 2

User Interface of the Experimental Chatbot (Mobile Version)

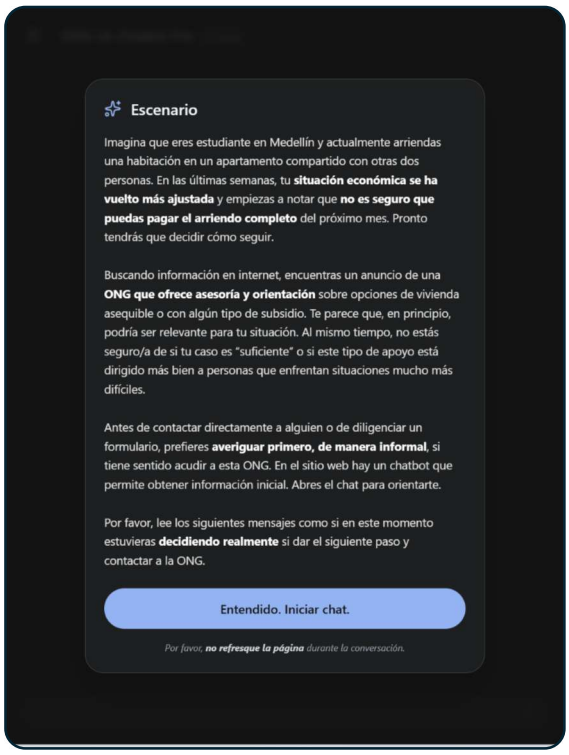


6. Attention Guidance and Anti-Satisficing Measures

Several mechanisms were implemented to encourage attentive processing of the chatbot interaction. Before the chat began, participants were presented with the scenario in a modal overlay. The scenario text was displayed progressively, and participants could only proceed once the display had completed, as shown in Figure 3.

Figure 3

Scenario Interface of the Experimental System



During chatbot message generation, response options were hidden. Only after the message had been fully displayed and the reading delay had elapsed were options shown.

At the end of the interaction, a completion modal explicitly indicated that the conversation had ended and required active confirmation before returning to the survey.

Together, these mechanisms reduce the likelihood of rapid skipping behavior and promote more deliberate engagement with the chatbot content.

7. Operationalization and Behavioral Metrics

Behavioral data were captured at each decision point within the interaction.

Table 4

Recorded Metrics

Metric	*Operationalization*
Reaction Time (RT)	Captured in milliseconds per dialogue step
Coded Choices	Internal identifiers for selected dialogue paths
Final Uptake	The definitive selection at the end of the intervention
Total Duration	Seconds elapsed from scenario start to final redirect

Reaction times were recorded using timestamps captured at the moment response options became visible and at the moment of user selection.

All data were transmitted back to Qualtrics via URL parameters and stored as embedded variables.

8. Source Code and Reproducibility

The complete source code used to implement the chatbot application is included as part of the supplementary materials and is publicly available via the GitHub Repository.

The repository contains all relevant components, including:

- React application files (*App.jsx, main.jsx*)
- dialogue scripts (experimental conditions)
- styling files (*index.css, App.css*)
- configuration files (*Vite, Tailwind, PostCSS*)

This ensures full transparency of the experimental implementation and allows exact replication of the digital environment.